

## 6.1 Lists ^

### Settings

#### Beta features

## container

A container is an object that groups related objects together.

## list / mutable

A list is a mutable container, meaning the size of the list can grow within the list can change.

### PARTICIPATION ACTIVITY

6.1.1: Lists contain references to other objects.

**Start** ☐ 2x speed

```
>>> my_list = ["hello", -4.2, 5]  
>>> |
```

Python (command line)

Python interpreter

```
my_list = ["hello"
```

| my_list |
|---------|
| "hello" |
| -4.2    |
| 5       |

## Captions ^

1. The user creates a new list.
2. The interpreter creates a new object for each list element.
3. "my\_list" holds references to objects in the list.

## list()

The `list()` function accepts a single iterable object argument, such and returns a new list object.

## index

An index is a zero-based integer matching to a specific position in elements.

Try 6.1.1: List's *i*th element can be directly accessed using `[i]`: Oldest people pr

Consider the following program that allows a user to print the age of the *N*th oldest known person are in a list sorted from oldest to youngest.

1. Modify the program to print the correct ordinal number ("1st", "2nd", "3rd", "4th", or "5th") instead
2. For the oldest person, remove the ordinal number (1st) from the print statement to say, "The ol

Reminder: List indices begin at 0, not 1, thus the print statement uses `oldest_people[nth_person - 1]` (element 1 at index 0, element 2 at index 1, etc.).



▶ Run

```
1  oldest_people = [122, 119, 117, 117, 116] # Source: Wikipedia.org
2
3  nth_person = int(input("Enter N (1-5): "))
4
5  if (nth_person >= 1) and (nth_person <= 5):
6      print(f"The {nth_person}th oldest person lived {oldest_people[nth_person - 1]} years old.")
7  # Your code goes here
```

CONSOLE**Model Solution** ▼

## in-place modification

Such growing and shrinking capability is called in-place modification.

Table 6.1.1: Some common list operations.

| Operation                        | Description                                                                                           | Example code                                                                                                   |
|----------------------------------|-------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| <code>my_list = [1, 2, 3]</code> | Creates a list.                                                                                       | <code>my_list = [1, 2, 3]</code><br><code>print(my_list)</code>                                                |
| <code>list(iter)</code>          | Creates a list.                                                                                       | <code>my_list = list('abc')</code><br><code>print(my_list)</code>                                              |
| <code>my_list[index]</code>      | Gets an element from a list.                                                                          | <code>my_list = [1, 2, 3]</code><br><code>print(my_list[1])</code>                                             |
| <code>my_list[start:end]</code>  | Gets a <i>new</i> list containing some of another list's elements.                                    | <code>my_list = [1, 2, 3, 4, 5]</code><br><code>print(my_list[1:4])</code>                                     |
| <code>my_list1 + my_list2</code> | Gets a <i>new</i> list with elements of <code>my_list2</code> added to end of <code>my_list1</code> . | <code>my_list1 = [1, 2, 3]</code><br><code>my_list2 = [4, 5]</code><br><code>print(my_list1 + my_list2)</code> |
| <code>my_list[i] = x</code>      | Changes the value of the element at index <code>i</code> in-place.                                    | <code>my_list = [1, 2, 3]</code><br><code>my_list[2] = 9</code><br><code>print(my_list)</code>                 |
| <code>del my_list[i]</code>      | Deletes the element from index <code>i</code> from a list.                                            | <code>my_list = [1, 2, 3]</code><br><code>del my_list[1]</code><br><code>print(my_list)</code>                 |

**PARTICIPATION  
ACTIVITY**

6.1.4: In-place modification of a list using indexing.

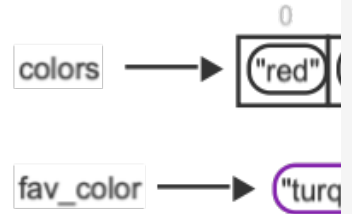
**Start**

2x speed

```
colors = ["red", "green", "blue"]

colors[1] = "yellow" # Change list element
print(colors)

fav_color = colors[2] # Bind to "blue"
fav_color = "turquoise" # List not altered
print(colors)
```



['red', 'yellow', 'b  
['red', 'yellow', 'b

Captions ^

1. A list can be modified in place by assigning a specific list index with a new value.
2. Assigning a variable with an element of an existing list, and then reassigning that variable modify the list.

## PythonTutor: In-place modification of a list.

```
→ my_teams = ["Raptors", "Heat", "Nets"]  
   your_teams = my_teams # Create a shared reference to  
  
   my_teams[1] = "Lakers" # Modify list element  
  
   print(f"My teams are: {my_teams}") # Both variables h  
   print(f"Your teams are: {your_teams}") # Both variabl
```

→ line that just executed

→ next line to execute



<< First

< Prev

Next >

Last >>

Step 1 of 5

Program output

Frames

Objects

## PythonTutor: In-place modification of a copy of a list.

```
→ my_teams = ["Raptors", "Heat", "Nets"]  
   your_teams = my_teams[:] # Assign your_teams with a CC  
  
   my_teams[1] = "Lakers" # Modify list element  
  
   print(f"My teams are: {my_teams}") # List element has  
   print(f"Your teams are: {your_teams}") # List element
```

→ line that just executed

→ next line to execute



<< First

< Prev

Next >

Last >>

Step 1 of 5

Program output

Frames

Objects

### CHALLENGE ACTIVITY

6.1.1: Enter the output for the list.

763650.1722066.qx3zqy7

**Start**

Type the program's output

```
user_values = [2, 5, 9]  
print(user_values)
```

1

2

3

4

5

**Check****Next**View solution  *(Instructors only)***CHALLENGE  
ACTIVITY**

## 6.1.2: Lists.

763650.1722066.qx3zqy7

**Start**

Eight values are read from input and stored in the list `list_data`. Output "List item: " followed by

[Click here for example](#) ▼

```
1 list_data = []
2 for elem in input().split():
3     if elem.isdigit(): # If elem is a digit, then convert to int.
4         list_data.append(int(elem))
5     else:
6         list_data.append(elem)
7
8 """ Your code goes here """
9
```

1

2

3

**Check****Next level**

View solution ▼ (Instructors only)